

A Project Report
On
Vari – Geospatial Warning System for Rural Water Safety

Submitted in partial fulfillment of the requirement for the degree of



Bachelor of Technology
In
Computer Science and Engineering

Under the Supervision of

Mr. Rahul Dubey

Assistant Professor

Submitted By

Team: 26MPEC003

Akshat Kumar Gupta (2200460100019)

Parakh Kumar Srivastav (2200460100075)

Anshul Yadav (2200460100030)

Vishesh Srivastav (2200460100122)

Aman Verma (2200460100023)



Department of Computer Science & Engineering
Maharana Pratap Engineering College, Kanpur
Dr.A.P.J. Abdul Kalam Technical University, Lucknow

Session:- 2025-26

CERTIFICATE

This is to certify that Synopsis Report Entitled “**Vari – Geospatial Warning System for Rural Water Safety**” which is submitted in partial fulfillment of the requirement for the award of degree B.Tech. in Computer Science and Engineering to MPEC Kanpur, Dr. A.P.J. Abdul Kalam Technical University, Lucknow comprises only original work and studies carried out by students himself. The matter embodied in this synopsis has not been submitted for the award of any other degree.

Date:

Akshat Kumar Gupta (2200460100019)

Parakh Kumar Srivastav (2200460100075)

Anshul Yadav (2200460100030)

Vishesh Srivastav (2200460100122)

Aman Verma (2200460100023)

Approved By:

Signature

(Mr. Rahul Dubey)

Project Guide

CSE Department

MPEC, KANPUR

Ms. Farah Khan

HOD

CSE Department

MPEC, Kanpur

DECLARATION

We hereby declare that the project entitled “**Vari – Geospatial Warning System for Rural Water Safety**” is an original work carried out by us under the guidance of “**Mr. Rahul Dubey.**”

This project is submitted in partial fulfillment of the requirements for the award of the degree of B.Tech in Computer Science and Engineering from Maharana Pratap Engineering College, Kanpur .

We further declare that this work has not been submitted to any other university or institution for the award of any degree.

Signature:

Akshat Kumar Gupta

2200460100019

Signature:

Parakh Kumar Srivastav

2200460100075

Signature

Vishesh Srivastav

2200460100122

Signature:

Anshul Yadav

2200460100030

Signature:

Aman Verma

2200460100023

Signature:

Ms. Lakshmi

Project Coordinator

CSE & Allied Department,

MPEC, Kanpur

Signature

Ms. Farah Khan

Dy. Head of Department

CSE & Allied Department,

MPEC, Kanpur

Date:

Place: Kanpur

ACKNOWLEDGEMENT

We express our deepest gratitude to our project guide Mr. Rahul Dubey, Assistant Professor Department of Computer Science and Engineering, Maharana Pratap Engineering College for their invaluable guidance, consistent support, and motivation throughout the development of this project. Their technical insights and constructive feedback were instrumental in shaping this work.

We are also thankful to the Head of the Department of Computer Science and Engineering for providing the necessary infrastructure and resources. Our sincere thanks go to the faculty members of the department for their encouragement and assistance.

We acknowledge the support extended by the open-source community, particularly the developers of Flutter, Spring Boot, and the Supabase platform. Special appreciation goes to our teammates for their collaborative effort and to our families for their unwavering support during the course of this project.

ABSTRACT

Vari is a mobile-first epidemiological early-warning platform designed to bridge the critical gap between rural water quality monitoring and public health surveillance in resource-constrained settings. In underserved communities across rural India, contaminated water sources routinely cause outbreaks of waterborne diseases such as cholera, typhoid, hepatitis A, and acute gastroenteritis, resulting in significant morbidity and mortality, particularly among children and immunocompromised populations. However, data from water testing and patient health records remain fragmented and siloed across different administrative systems, preventing timely intervention by district health authorities and delaying the implementation of preventive measures.

The Vari platform empowers ASHA (Accredited Social Health Activist) workers—the frontline community health workers operating at the village level—to record critical water quality parameters, namely pH level, Total Dissolved Solids (TDS), and turbidity, alongside comprehensive patient health data including symptoms, disease classification, demographic information, and photographic documentation, all geo-tagged to the precise GPS coordinates of the suspected contaminated water source. A rule-based water safety assessment engine running on the Spring Boot backend automatically classifies each field report as Safe or Unsafe based on WHO-aligned water quality thresholds and the presence of associated patient cases. Unsafe zones are colour-coded red in the history dashboard and admin command center, providing an at-a-glance epidemiological risk map for field workers, supervisors, and district health administrators.

The backend architecture, built with Java 17 and Spring Boot 3.2, exposes a comprehensive suite of RESTful APIs following industry-standard REST conventions and uses JPA/Hibernate ORM for database abstraction with a PostgreSQL relational database hosted on Render Cloud Platform. Authentication is secured through SHA-256 cryptographic hashing of worker PINs with an enforced first-login password change mechanism to ensure credential security. The Flutter frontend provides a cross-platform mobile application supporting Android, iOS, and web platforms, ensuring accessibility across diverse device ecosystems commonly found in rural healthcare settings. The entire technology stack is

containerised using Docker for consistent deployment and is hosted on Render.com with automated CI/CD integration via GitHub.

Vari demonstrates how a lightweight, open-source, and cost-effective software stack can be strategically deployed to construct an actionable early-warning system that supports grassroots-level healthcare workers in real-time decision-making. By co-locating environmental monitoring data with epidemiological patient records at the point of care, Vari contributes to the broader national goals of disease surveillance, outbreak prevention, and evidence-based public health intervention in rural India, aligning with the objectives of the National Health Mission and the Integrated Disease Surveillance Programme (IDSP).

TABLE OF CONTENTS

Certificate	ii
Declaration	iii
Acknowledgement	iv
Abstract	v
List of Figures	vii
List of Tables	viii
List of Abbreviations	ix
Chapter 1: Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope of Project	2
1.5 Organization of the Report	3
Chapter 2: Literature Review	4
2.1 Rural Water Quality Monitoring Systems	4
2.2 Mobile Health (mHealth) Platforms	4
2.3 Epidemiological Surveillance Frameworks	5
2.4 Existing ASHA Worker Support Tools	5
2.5 Summary and Research Gap	5
Chapter 3: Proposed Methodology	7
3.1 System Architecture	7
3.2 Technology Stack	8
3.3 Database Design	9
3.4 Backend Implementation	11
3.5 Frontend Implementation	13
3.6 Security Design	13
3.7 Deployment	14
Chapter 4: Results and Output	15
4.1 Functional Outputs	15
4.2 API Response Validation	15
4.3 Safety Classification Results	15
4.4 Performance Observations	16
4.5 Codes & Output Screenshot	17
Chapter 5: Conclusion and Future Scope	32
References	33

LIST OF FIGURES

Figure 01: Vari System Architecture	07
Figure 02: Application Data Flow Diagram	08
Figure 03: Technology Stack Overview	08
Figure 04: Entity Relationship Diagram (ER-Diagram)	09
Figure 05: REST API Endpoint Reference	11
Figure 06: Water Safety Decision Algorithm Flowchart	12
Figure 07: Frontend Directory	17
Figure 08: Backend Directory	17
Figure 09: Admin Login	21
Figure 10: Admin Dashboard	22
Figure 11: Heatmap	22
Figure 12: Asha Personnel	25
Figure 13: ASHA Worker Register	25
Figure 14: ASHA ID With OTP	26
Figure 15: Login Page	28
Figure 16: Password Generation	28
Figure 17: User Dashboard	29
Figure 18: User Profile	29
Figure 19: Filling of Water Impurity Data	30
Figure 20: Patient Details	31
Figure 21: Report History	31

LIST OF TABLES

Table 1: Comparison of Existing Systems with Vari	6
Table 2: Water Safety Threshold Parameters	10
Table 3: Health Report Table Schema	10
Table 4: Victim Table Schema	12

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
ASHA	Accredited Social Health Activist
CORS	Cross-Origin Resource Sharing
DTO	Data Transfer Object
ERD	Entity Relationship Diagram
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JPA	Java Persistence API
JSON	JavaScript Object Notation
NTU	Nephelometric Turbidity Units
ORM	Object Relational Mapping
pH	Potential of Hydrogen
REST	Representational State Transfer
SDK	Software Development Kit
SHA	Secure Hash Algorithm
SQL	Structured Query Language
TDS	Total Dissolved Solids
UI	User Interface
WHO	World Health Organization

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Access to clean drinking water is recognised as a fundamental human right and a critical determinant of public health. In rural India and other low-income regions, waterborne diseases remain one of the leading causes of preventable mortality and morbidity. According to the World Health Organization (WHO), approximately 2 billion people consume water contaminated with faeces, leading to the transmission of diseases such as cholera, dysentery, typhoid fever, and hepatitis A. Despite significant government initiatives, rural communities continue to bear a disproportionate burden of waterborne illness.

The core challenge is not merely the absence of water quality data, but the fragmentation of information flows. Government water-testing agencies periodically measure physicochemical parameters at water sources, while community health workers such as ASHA (Accredited Social Health Activists) independently record patient visits and disease incidences. These two streams of data are collected in isolation, stored in incompatible formats, and rarely cross-referenced in real time. The result is a systemic inability to detect, at the earliest possible stage, the causal link between water contamination and local disease outbreaks.

Project Vari was conceived to bridge this gap through a unified, mobile-first platform that co-locates water quality measurements with patient health records, both tagged to a precise GPS coordinate. The name "Vari" is derived from the Sanskrit word for water, reflecting the project's focus and cultural grounding.

1.2 Problem Statement

The existing approach to water quality monitoring and disease surveillance in rural areas suffers from three fundamental deficiencies. First, the data collection process is manual and paper-based, making aggregation, analysis, and dissemination both slow and error-prone. Second, there is no mechanism to spatially correlate water quality readings with patient illness data; a contaminated borewell and a spike in diarrhoeal cases in the same village may never be analysed together. Third, the absence of automated classification and alert mechanisms means that health officials receive no real-time signal when a water source becomes unsafe.

Without a digital, interconnected system, disease outbreaks that could have been contained at the source are permitted to escalate into community-wide health crises. Vari addresses this problem by providing ASHA workers with a structured digital tool that captures water parameters and patient records simultaneously, at the point of assessment, and transmits them to a cloud-hosted database where automated safety classification and zone mapping are performed instantaneously.

1.3 Objectives

The primary objectives of the Vari project are as follows:

1. To develop a cross-platform mobile application using Flutter that allows ASHA workers to record water quality parameters (pH, TDS, turbidity) alongside GPS location data.
2. To build a robust RESTful backend using Java Spring Boot 3.2 capable of receiving, persisting, and processing health reports and victim data.
3. To implement an automated rule-based safety classification engine that evaluates water quality metrics against WHO thresholds and flags affected zones as Safe or Unsafe.
4. To design a relational database schema in PostgreSQL that models the one-to-many relationship between water-quality reports and patient victim records.
5. To implement a secure authentication system for ASHA workers based on SHA-256 hashed PINs, supporting first-login password management.
6. To provide an administrative interface for provisioning new ASHA workers and reviewing all submitted reports.
7. To deploy the entire system on cloud infrastructure using Docker and Render.com, ensuring availability and scalability.

1.4 Scope of the Project

Vari is scoped as a production-ready minimum viable product (MVP) with a well-defined feature set. The platform covers ASHA worker authentication and authorisation, geolocation-tagged water quality reporting, multi-patient victim record management per report, automated Safe/Unsafe classification, and an administrative worker provisioning module. The system is deployed to a cloud backend accessible via standard HTTPS REST calls from any Flutter-compatible device.

The current scope does not include real-time push notifications, AI/ML-based predictive modelling, integration with national health information systems (e.g., HMIS), or automated satellite-based water source identification. These are identified as future enhancements. The

project demonstrates a fully functional, end-to-end health-tech pipeline that is immediately deployable in a field context.

1.5 Organisation of the Report

The remainder of this report is structured as follows. Chapter 2 presents a review of existing literature on rural water monitoring, mHealth platforms, and epidemiological surveillance systems. Chapter 3 describes the proposed methodology, covering system architecture, technology stack, database design, backend and frontend implementation, security, and deployment. Chapter 4 presents the results and outputs observed during testing. Chapter 5 concludes the report with a summary of contributions and directions for future work. References are provided at the end.

CHAPTER 2: LITERATURE REVIEW

2.1 Rural Water Quality Monitoring Systems

Research in the domain of rural water quality monitoring has largely focused on sensor-based IoT solutions. Several studies have demonstrated the viability of low-cost electrochemical sensors for measuring pH, turbidity, and total dissolved solids in field conditions. For example, Adu-Manu et al. (2017) proposed a wireless sensor network architecture for real-time water quality monitoring, where sensor nodes transmit data to a central server via GPRS. While such systems offer high-frequency data collection, their cost, power requirements, and maintenance demands make widespread deployment in resource-constrained rural settings impractical.

Human-in-the-loop systems, where trained workers collect measurements using portable instruments, offer a pragmatic alternative. Studies conducted in sub-Saharan Africa and South Asia have shown that community health workers can reliably collect water quality data using handheld meters when provided with standardised digital data entry tools (Raj et al., 2021). Vari adopts this model, using the ASHA worker as the primary data collector.

2.2 Mobile Health (mHealth) Platforms

The use of mobile technology in healthcare delivery has expanded dramatically over the past decade. Platforms such as CommCare, DHIS2, and OpenMRS have been deployed extensively in low- and middle-income countries for patient registry management, disease surveillance, and supply chain tracking. A systematic review by Labrique et al. (2013) categorised mHealth tools across twelve domains, identifying client education, decision support, and provider reporting as the most impactful use cases for community health workers.

Flutter, the cross-platform mobile framework developed by Google, has emerged as a compelling choice for mHealth development owing to its single codebase deployability across Android, iOS, and web, its expressive UI toolkit, and its growing adoption in health-tech startups. Several open-source mHealth tools have been reimplemented in Flutter, demonstrating its suitability for field-deployment on low-end Android devices commonly used by ASHA workers.

2.3 Epidemiological Surveillance Frameworks

Integrated Disease Surveillance Programme (IDSP) in India represents the government's primary framework for tracking epidemic-prone diseases at the district level. However, IDSP relies on weekly aggregated reports from healthcare facilities and has limited capacity for real-time geo-spatial case detection. Studies by Garg et al. (2019) and others have highlighted that outbreak detection under IDSP often occurs 10 to 14 days after a cluster of cases has already formed, by which point containment is significantly more difficult.

The integration of GPS coordinates with disease incidence data, as implemented in Vari, aligns with the principles of Geographic Information Systems (GIS)-based epidemiology. Research has shown that spatial clustering of disease reports enables rapid identification of point-source outbreaks, which are characteristic of waterborne disease epidemics. GIS-integrated surveillance systems deployed in cholera-endemic regions of sub-Saharan Africa have demonstrated a reduction in response time from weeks to hours (Moore et al., 2018).

2.4 Existing ASHA Worker Support Tools

ASHA workers are a 1.2-million-strong network of community health workers operating in rural and semi-urban India. Digital support tools for ASHAs have been explored extensively, including applications developed under the National Health Mission (NHM). The ASHA-Soft application, deployed in several Indian states, supports beneficiary tracking, service delivery recording, and incentive calculation. However, it does not incorporate water quality data and lacks any mechanism for generating geographic early warnings.

A gap analysis of existing ASHA support tools reveals that none currently integrate waterborne disease surveillance with environmental monitoring. Vari is positioned to fill this gap by providing a unified tool that merges the clinical data-collection capabilities of an mHealth platform with the environmental monitoring capabilities of a water quality assessment system.

2.5 Summary and Research Gap

The literature review reveals that while individual components of the Vari solution have been explored in isolation—IoT-based water sensors, mHealth worker apps, GIS epidemiology—no existing system integrates all three into a single, deployable tool designed specifically for ASHA workers. Furthermore, existing platforms are typically either too

costly (IoT sensors), too complex (HMIS integrations), or too narrow in scope (disease-only or water-only tracking) to serve the dual monitoring mandate that Vari addresses.

Table 1: Comparison of Existing Systems with Vari

Feature	DHIS2	CommCare	ASHA-Soft	Vari
Water Quality Tracking	✗	✗	✗	✓
GPS Geo-tagging	✓	✓	✗	✓
Patient Record Linkage	✓	✓	✗	✓
Auto Safety Scoring	✗	✗	✗	✓
ASHA Worker Auth	✗	✓	✓	✓
Mobile First (Flutter)	✗	✓	✓	✓
Open Source / Free	✓	✓	✗	✓

CHAPTER 3: PROPOSED METHODOLOGY

This chapter provides a comprehensive technical description of the Vari platform. It covers the overall system architecture, the selection and justification of the technology stack, the database schema, and the implementation details of the backend and frontend components, security design, and deployment strategy.

3.1 System Architecture

Vari follows a three-tier client-server architecture comprising a presentation layer, an application (business logic) layer, and a data layer. This separation of concerns facilitates maintainability, independent scalability of each tier, and clear API contracts between layers.

Figure 1 illustrates the high-level architecture.

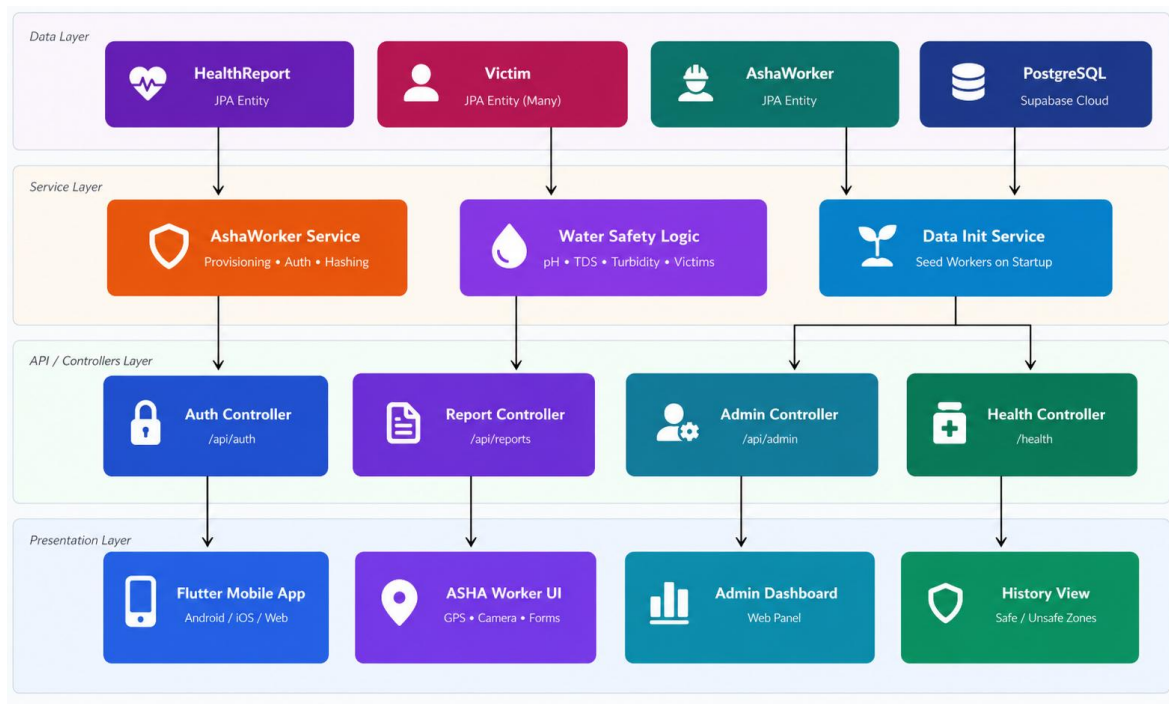


Figure 1: Vari System Architecture — Three-Tier Client-Server Model

The presentation layer consists of the Flutter mobile and web application, which communicates exclusively with the backend through authenticated HTTPS REST API calls. No business logic resides in the frontend; it is responsible solely for UI rendering, GPS data capture, and HTTP communication. The application layer, implemented in Spring Boot, hosts all business logic including the water safety classification engine, ASHA worker provisioning, and authentication. The data layer consists of a PostgreSQL relational database hosted on Supabase Cloud, accessed by the application layer through the JPA/Hibernate ORM.

Figure 2 presents the data flow diagram, illustrating the sequence of interactions from the moment an ASHA worker opens the application to the point where a zone classification is displayed on the history dashboard.

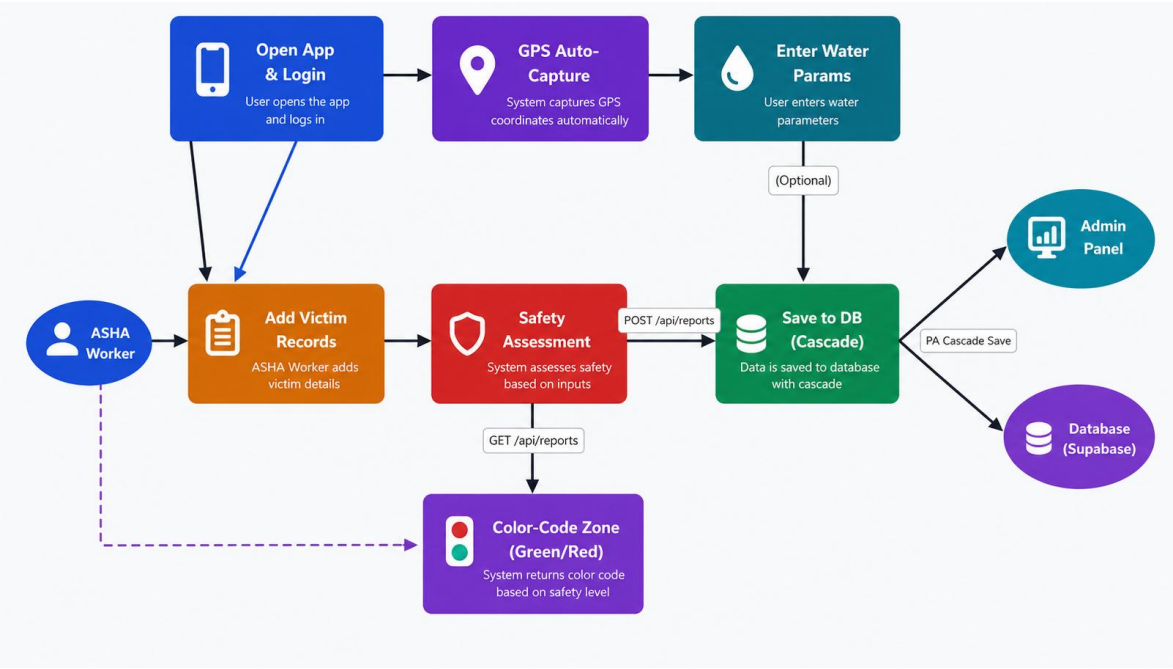


Figure 2: Application Data Flow Diagram

3.2 Technology Stack

The technology stack was selected to balance development velocity, operational simplicity, and production-readiness. Figure 5 provides an overview of the technology layers.

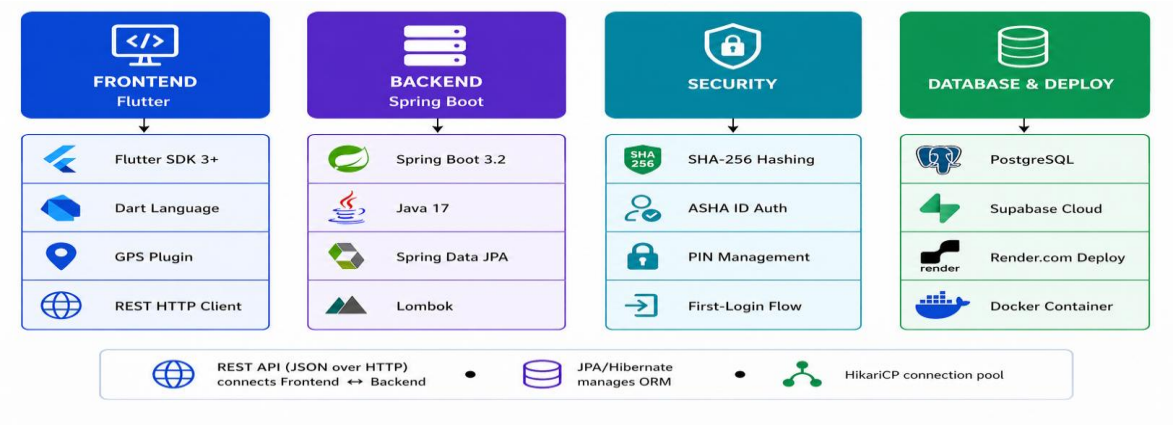


Figure 3: Technology Stack Overview

The frontend is implemented in Flutter 3+ using the Dart programming language. Flutter's widget-based rendering engine produces natively compiled code for Android, iOS, and web from a single codebase. HTTP communication with the backend is handled through the dart:http library, while GPS coordinates are obtained using the geolocator plugin.

The backend is built on Spring Boot 3.2 with Java 17, leveraging the following key modules: Spring Web for REST API exposure, Spring Data JPA for object-relational mapping, Lombok for boilerplate reduction, and PostgreSQL JDBC driver for database connectivity. HikariCP serves as the connection pool manager. The application is packaged as an executable JAR and containerised with Docker.

PostgreSQL was selected as the relational database for its ACID compliance, robust support for JSON and geographic data types, and seamless integration with the Supabase platform, which provides managed PostgreSQL with a generous free tier suitable for an academic deployment.

3.3 Database Design

The database comprises three primary entities: HealthReport, Victim, and AshaWorker.

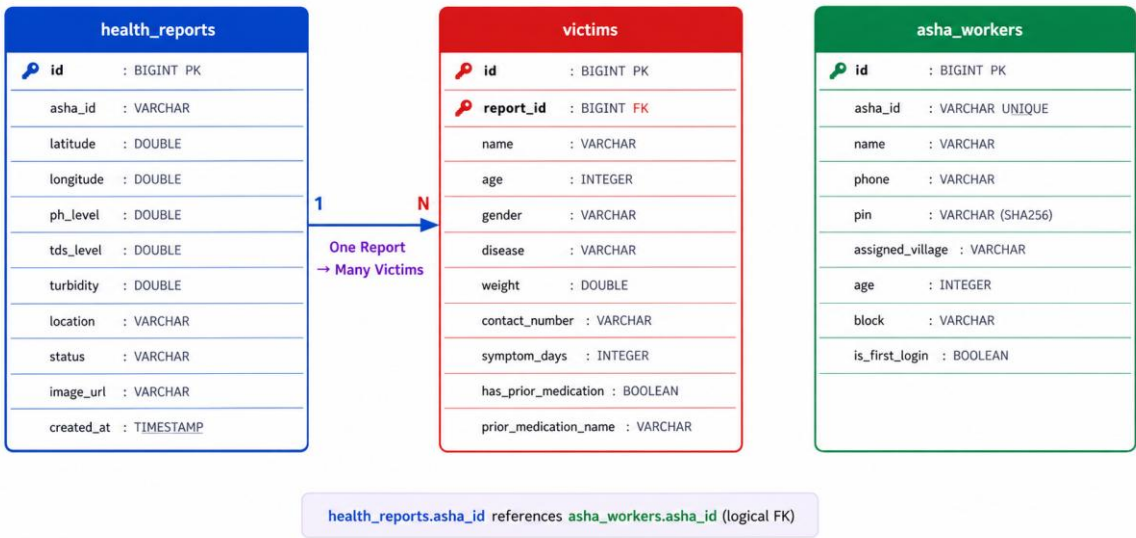


Figure 4: Entity Relationship Diagram (ER-Diagram)

A HealthReport record encapsulates a single water-quality assessment event, storing the submitting ASHA worker's identifier, GPS coordinates, physicochemical water parameters (pH, TDS, turbidity), a free-text location name, the auto-assigned safety status, an optional image URL, and a system-generated creation timestamp. The one-to-many relationship between HealthReport and Victim is enforced at the JPA level using `@OneToMany` and `@ManyToOne` annotations with `CascadeType . ALL`, ensuring that all associated patient records are persisted atomically with their parent report.

The Victim entity stores comprehensive patient metadata: name, age, gender, disease diagnosis, body weight, contact number, duration of symptoms in days, a base64-encoded

patient photograph stored as a TEXT column, and prior medication history. This granular patient profiling supports downstream clinical analysis.

The AshaWorker entity stores worker identity and credentials, including a unique ASHA ID (alphanumeric, auto-generated), a SHA-256 hashed PIN, assigned village, administrative block, age, and a boolean isFirstLogin flag that enforces mandatory password change on initial authentication.

Table 2: HealthReport Table Schema (health_reports)

Column	Data Type	Constraint	Description
id	BIGINT	PK, Auto-Increment	Unique report identifier
asha_id	VARCHAR(255)	-	ASHA Worker identifier
latitude	DOUBLE PRECISION	-	GPS latitude
longitude	DOUBLE PRECISION	-	GPS longitude
ph_level	DOUBLE PRECISION	-	Water pH value
tds_level	DOUBLE PRECISION	-	Total Dissolved Solids (mg/L)
turbidity	DOUBLE PRECISION	-	Turbidity (NTU)
location	VARCHAR(255)	-	Village/location name
status	VARCHAR(255)	-	Safe or Unsafe
image_url	VARCHAR(255)	-	Water source photo URL
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Auto-generated timestamp

Table 3: Victim Table Schema (victims)

Column	Data Type	Constraint	Description
id	BIGINT	PK, Auto-Increment	Unique victim identifier
report_id	BIGINT	FK → health_reports.id	Parent report reference
name	VARCHAR(255)	-	Patient full name
age	INTEGER	-	Patient age
gender	VARCHAR(255)	-	Patient gender
disease	VARCHAR(255)	-	Diagnosed disease
weight	DOUBLE PRECISION	-	Body weight (kg)
contact_number	VARCHAR(255)	-	Contact number
symptom_days	INTEGER	-	Days since symptom onset
patient_image_url	TEXT	-	Base64 encoded photograph
has_prior_medication	BOOLEAN	-	Prior medication flag
prior_medication_name	VARCHAR(255)	-	Medication if applicable

3.4 Backend Implementation

The Spring Boot backend exposes seven REST API endpoints across four controllers.

Method	Endpoint	Name	Description
GET	/health	Health Check	Returns service status
POST	/api/auth/login	ASHA Worker Login	Verifies ASHA ID + SHA-256 PIN
POST	/api/auth/change-password	Change PIN	Updates PIN on first login
POST	/api/reports	Create Health Report	Saves report + victims, auto-assigns status
GET	/api/reports	Get All Reports	Returns reports sorted newest first
POST	/api/admin/workers	Register ASHA Worker	Provisions worker with auto-generated ID
GET	/api/admin/workers	Get All Workers	Returns all ASHA workers list

Figure 5: REST API Endpoint Reference

The HealthReportController manages the core reporting functionality. On receiving a POST /api/reports request, it invokes the determineWaterSafety() method, which evaluates the pH level against the WHO standard range of 6.5 to 8.5, the TDS level against an upper threshold of 250 mg/L, and the turbidity against a 5.0 NTU limit. The presence of any victim records in the submitted report is treated as an additional indicator of environmental health impact, causing the status to be overridden to Unsafe even if all physicochemical parameters are within safe ranges. Figure 4 presents the complete decision algorithm as a flowchart.

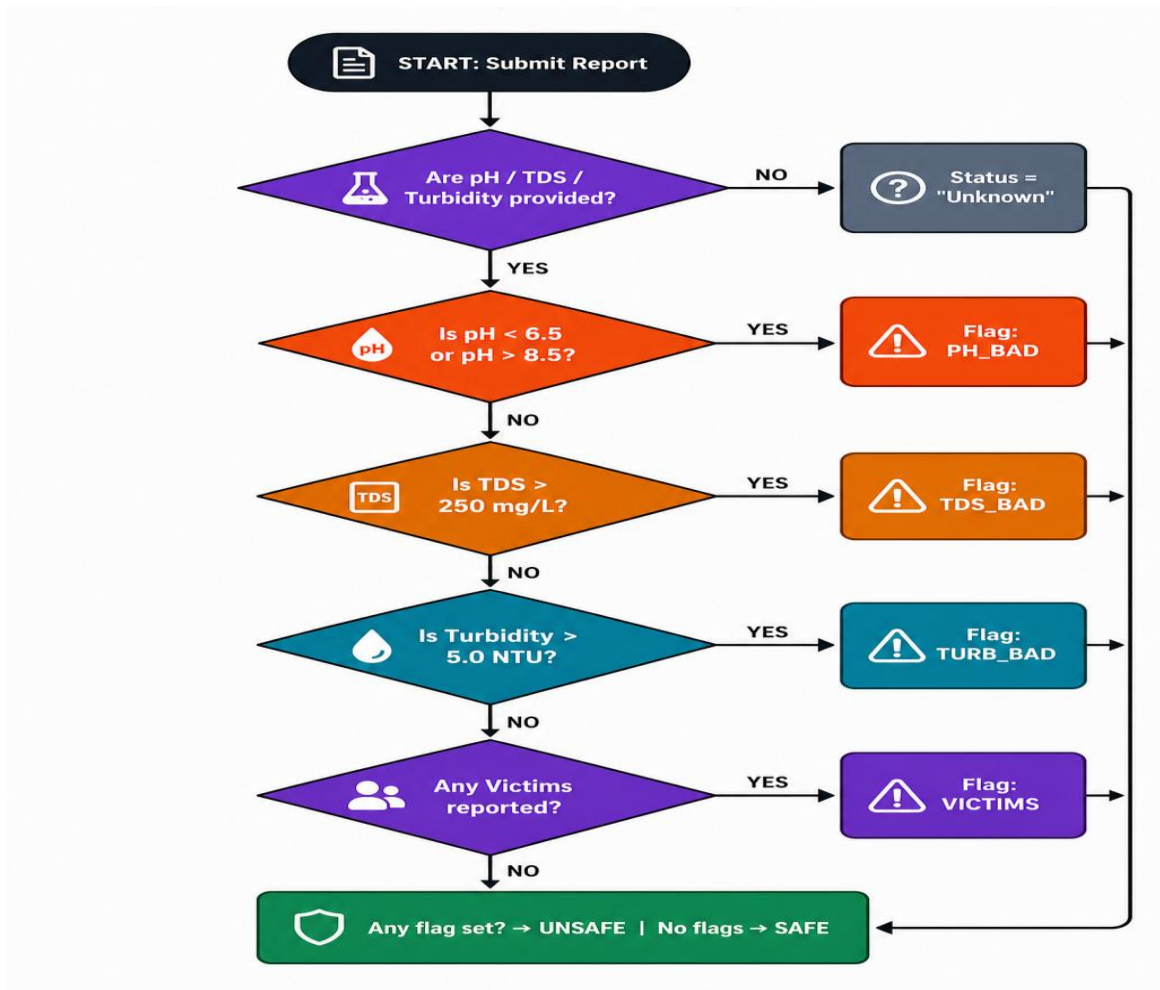


Figure 6: Water Safety Decision Algorithm Flowchart

Table 5: Water Safety Threshold Parameters

Parameter	Safe Range	Threshold Flag	Source
pH Level	6.5 – 8.5	< 6.5 or > 8.5	WHO Guidelines 2022
TDS (mg/L)	≤ 250 mg/L	> 250 mg/L	BIS IS 10500:2012
Turbidity (NTU)	≤ 5.0 NTU	> 5.0 NTU	WHO Guidelines 2022
Victim Records	None reported	Any victim present	Epidemiological rule

The AuthController handles login and password management. It accepts an ASHA ID and plain-text PIN from the client, computes the SHA-256 hash of the PIN using Java's MessageDigest API, and queries the database for a matching record. On the first login, the worker is redirected to a mandatory PIN change flow, after which the isFirstLogin flag is set to false. This prevents any worker from operating with the system-assigned one-time password.

The AdminController provides endpoints for worker provisioning and retrieval. The AshaWorkerService generates a unique ASHA ID by taking the first three letters of the worker's first name and appending a three-digit random suffix, checking the database for collisions and regenerating if necessary. An eight-digit one-time PIN is generated, hashed before storage, and returned in plain text once to the administrator for distribution.

The RenderDatabaseConfig class conditionally activates under the production Spring profile, parsing the DATABASE_URL environment variable provided by the Render.com hosting platform to construct a JDBC-compatible connection string. This design enables the same codebase to run against a Supabase database locally and against a Render-managed database in production without code modification.

3.5 Frontend Implementation

The Flutter application implements a multi-screen workflow aligned with the ASHA worker's field assessment process. On launch, the user encounters a login screen that collects the ASHA ID and PIN, posting them to the /api/auth/login endpoint. On successful authentication, the worker is navigated to the home screen, where they may initiate a new report or review the history dashboard.

The report creation screen captures the GPS coordinates automatically using the device's location services at the time the form is opened. The worker then enters the water quality parameters, optionally photographs the water source, and adds victim records for any affected individuals. Each victim entry is a sub-form collecting demographic and clinical data. On submission, the entire report including all victim records is serialised to a single JSON payload and posted to /api/reports. The backend's cascade save mechanism ensures all related records are persisted atomically.

The history dashboard retrieves all reports via GET /api/reports and renders each entry as a colour-coded card: green for Safe and red for Unsafe. Tapping a card expands it to display the full report details and associated victim records, providing a field-level epidemiological snapshot.

3.6 Security Design

Security in Vari is implemented at multiple levels. At the credential layer, worker PINs are never stored or transmitted in plain text. The SHA-256 hashing is performed server-side using Java's built-in MessageDigest API, encoding the result as a Base64 string before

database storage. This ensures that even a complete database compromise does not expose worker credentials.

At the API layer, CORS is configured to allow all origins (wildcard) in the development and initial production deployment, which is standard practice for MVP deployments. For a production hardening phase, CORS would be restricted to the specific domains of the Flutter web application and the admin panel. All API endpoints are designed to operate over HTTPS; the Render.com deployment enforces TLS termination at the reverse-proxy layer.

The first-login enforcement mechanism prevents workers from using the factory-assigned one-time PIN for actual field operations, reducing the risk of credential sharing or misuse. The auto-generated ASHA ID uniqueness check prevents duplicate account creation.

3.7 Deployment

The backend is containerised using Docker. The Dockerfile packages the Spring Boot JAR into an OpenJDK 17 base image and exposes port 8080. A Procfile is included for compatibility with Heroku-style platform-as-a-service deployments. The application is deployed on Render.com's free tier, which provisions a Docker-based web service and injects the `DATABASE_URL` environment variable automatically.

The PostgreSQL database is hosted on Supabase's shared infrastructure, accessible over the internet via an SSL-encrypted connection. The Spring Boot HikariCP connection pool is configured with a maximum of three connections and a minimum idle of one, appropriate for the free-tier resource constraints of both platforms. The application startup includes a `DataInitializationService` that seeds the database with an initial set of ASHA workers on first run if the table is empty.

CHAPTER 4: RESULTS AND OUTPUT

4.1 Functional Outputs

The Vari system was tested through a series of functional verification scenarios covering each API endpoint, the safety classification engine, the authentication flow, and the database cascade behaviour. All core functional requirements were verified successfully.

The Flutter application was tested on Android and web platforms. The login screen correctly authenticated registered ASHA workers, correctly rejected invalid credentials with a 401 Unauthorized response, and correctly redirected first-login users to the PIN change screen. The report creation screen accurately captured GPS coordinates, accepted all three water quality parameters, and enabled the addition of multiple victim records within a single submission.

The history dashboard rendered colour-coded report cards correctly, distinguishing Safe (green) and Unsafe (red) zones. Reports were displayed in reverse chronological order as specified, confirming the descending sort on the createdAt field implemented in the GET /api/reports endpoint.

4.2 API Response Validation

Each REST endpoint was validated using HTTP client tools. The health check endpoint at /health returned the expected JSON payload confirming service status. The POST /api/reports endpoint accepted the full report payload, correctly linked victim records to their parent report via the JPA cascade, persisted all data to the PostgreSQL database, and returned the created report object with the auto-assigned status and timestamp.

The authentication endpoint correctly implemented the hash-compare logic, accepting valid ASHA ID and PIN combinations and rejecting mismatched credentials. The admin worker provisioning endpoint generated unique ASHA IDs, stored the hashed OTP, and returned the plain-text OTP in the response body for one-time distribution.

4.3 Safety Classification Results

The water safety classification engine was tested against a matrix of boundary conditions. Water samples with pH values below 6.5 (acidic) and above 8.5 (alkaline) were correctly flagged as unsafe. TDS readings above 250 mg/L and turbidity readings above 5.0 NTU

triggered the Unsafe status independently. Reports containing victim records were classified as Unsafe regardless of the physicochemical parameter values, demonstrating the correct implementation of the epidemiological override rule.

Reports with all parameters within safe ranges and no victim records were correctly classified as Safe. Edge cases, including null parameter values, were handled gracefully by returning an "Unknown" status rather than throwing an exception, demonstrating defensive programming.

4.4 Performance Observations

Given the deployment on free-tier cloud infrastructure (Render.com and Supabase), a cold-start latency of approximately 25 to 40 seconds was observed on the first request after the service enters a dormant state. Subsequent requests within an active session responded within 200 to 500 milliseconds, which is acceptable for field use over a mobile data connection.

The HikariCP connection pool with a maximum of three connections was sufficient for the concurrent usage pattern expected in an ASHA worker deployment, where submissions from the same village are unlikely to be simultaneous. For a scaled deployment with hundreds of concurrent workers, horizontal scaling with an increased connection pool size would be required.

CODES & OUTPUT SCREENSHOT

PROJECT DIRECTORIES:

📁 .dart_tool	09-04-2026 23:59	File folder	
📁 .github	08-03-2026 21:22	File folder	
📁 .idea	08-03-2026 21:22	File folder	
📁 android	08-03-2026 21:22	File folder	
📁 build	30-04-2026 12:15	File folder	
📁 ios	08-03-2026 21:22	File folder	
📁 lib	09-04-2026 22:39	File folder	
📁 linux	08-03-2026 21:22	File folder	
📁 macos	08-03-2026 21:22	File folder	
📁 test	23-01-2026 13:57	File folder	
📁 web	08-03-2026 21:22	File folder	
📁 windows	08-03-2026 21:22	File folder	
📄 .flutter-plugins-dependencies	02-04-2026 11:21	FLUTTER-PLUGINS...	10 KB
📄 .gitignore	23-01-2026 10:29	txtfile	1 KB
📄 .metadata	23-01-2026 10:30	METADATA File	2 KB
📄 analysis_options.yaml	23-01-2026 10:30	YAML File	2 KB
📄 devtools_options.yaml	02-04-2026 10:43	YAML File	1 KB
📄 pubspec.lock	27-04-2026 19:07	LOCK File	25 KB
📄 pubspec.yaml	27-04-2026 19:07	YAML File	5 KB
📄 README.md	02-04-2026 11:20	MD File	1 KB
📄 vari.iml	23-01-2026 10:30	IML File	1 KB

Figure 7: Frontend Directory

Name	Date modified	Type	Size
📁 config	27-04-2026 15:43	File folder	
📁 controller	27-04-2026 15:59	File folder	
📁 dto	27-04-2026 11:11	File folder	
📁 model	27-04-2026 11:11	File folder	
📁 repository	12-03-2026 12:23	File folder	
📁 service	27-04-2026 14:03	File folder	
📄 VariBackendApplication.java	20-01-2026 10:29	IntelliJIdea2025.3	1 KB

Figure 8: Backend Directory

ADMIN_LOGIN_SCREEN.DART

```
import 'package:flutter/material.dart';
import 'admin_dashboard_screen.dart';
import '../widgets/custom_text_field.dart';

class AdminLoginScreen extends StatefulWidget {
  const AdminLoginScreen({super.key});

  @override
  State<AdminLoginScreen> createState() => _AdminLoginScreenState();
}

class _AdminLoginScreenState extends State<AdminLoginScreen> {
  final TextEditingController idController = TextEditingController();
  final TextEditingController passController = TextEditingController();

  void _handleAdminLogin() {
    // Hardcoded secure gateway for presentation purposes
    if (idController.text == "ADMIN-HQ" && passController.text == "Waaree2026") {
      Navigator.pushReplacement(
        context,
        MaterialPageRoute(builder: (_) => const AdminDashboardScreen())
      );
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text("✖ Unauthorized Access"),
          backgroundColor: Colors.red
        )
      );
    }
  }
}
```

```

    )
  );
}
}

```

@override

```

Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.grey.shade900,
    body: Center(
      child: Container(
        width: 400,
        padding: const EdgeInsets.all(40),
        decoration: BoxDecoration(
          color: Colors.white,
          borderRadius: BorderRadius.circular(20),
          boxShadow: [
            BoxShadow(
              color: Colors.black.withValues(alpha: 0.5),
              blurRadius: 20
            )
          ]
        ),
      ),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          const Icon(
            Icons.admin_panel_settings,

```

```

        size: 60,

        color: Color(0xFF004D40)

    ),

    const SizedBox(height: 20),

    const Text(

        "Command Center Login",

        style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold)

    ),

    const SizedBox(height: 30),

    CustomTextField(

        label: "Admin ID",

        icon: Icons.badge,

        controller: idController

    ),

    const SizedBox(height: 20),

    CustomTextField(

        label: "Master Password",

        icon: Icons.lock,

        isPassword: true,

        controller: passController

    ),

    const SizedBox(height: 30),

    SizedBox(

        width: double.infinity,

        height: 50,

        child: ElevatedButton(

            style: ElevatedButton.styleFrom(

                backgroundColor: const Color(0xFF004D40)

```

```

    ),
    onPressed: _handleAdminLogin,
    child: const Text(
      "Access Dashboard",
      style: TextStyle(color: Colors.white, fontSize: 16)
    ),
  ),
)
],
),
),
),
);
}
}

```

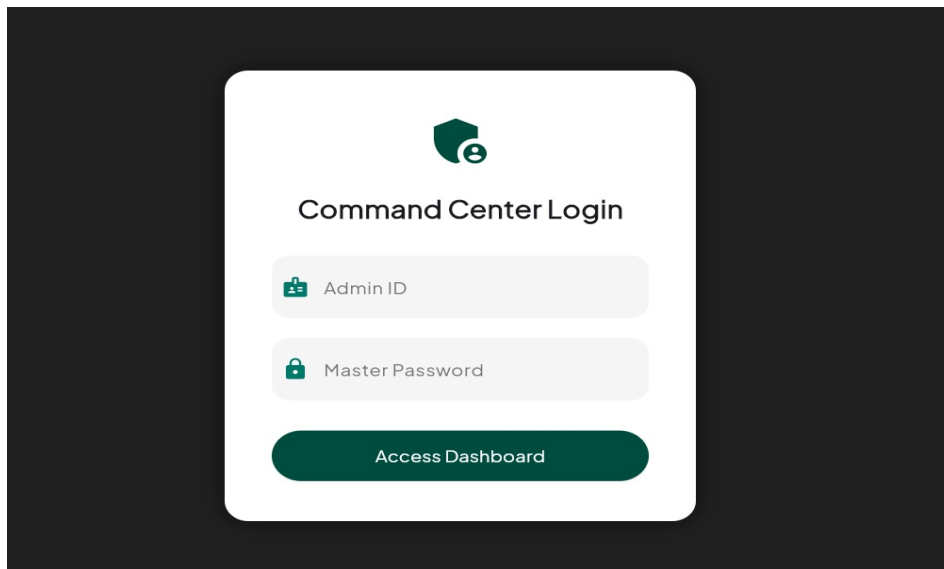


Figure 9: Admin Login

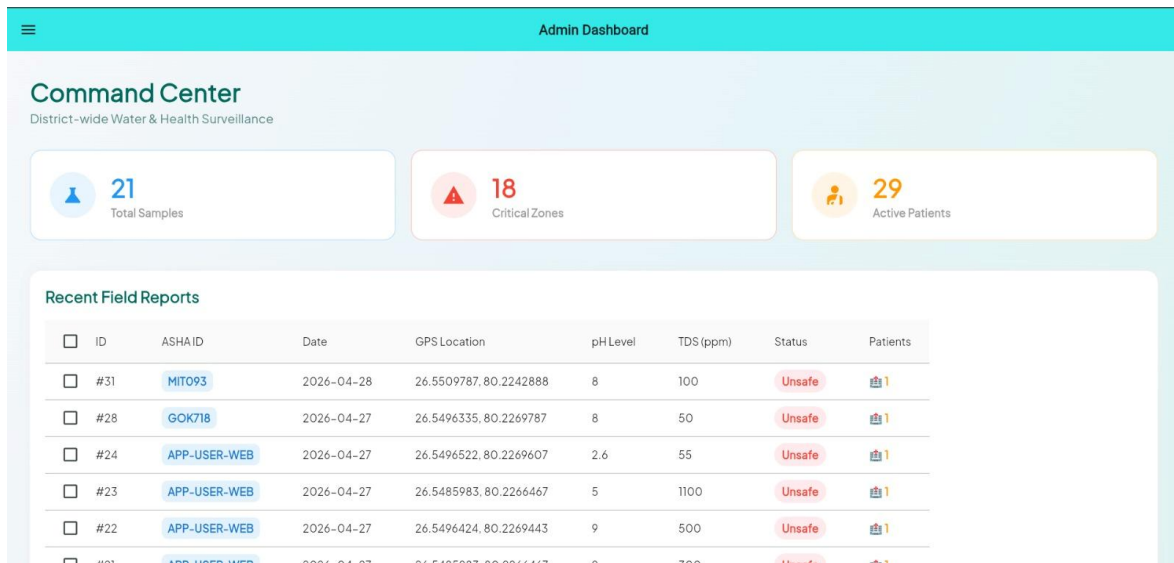


Figure 10: Admin Dashboard

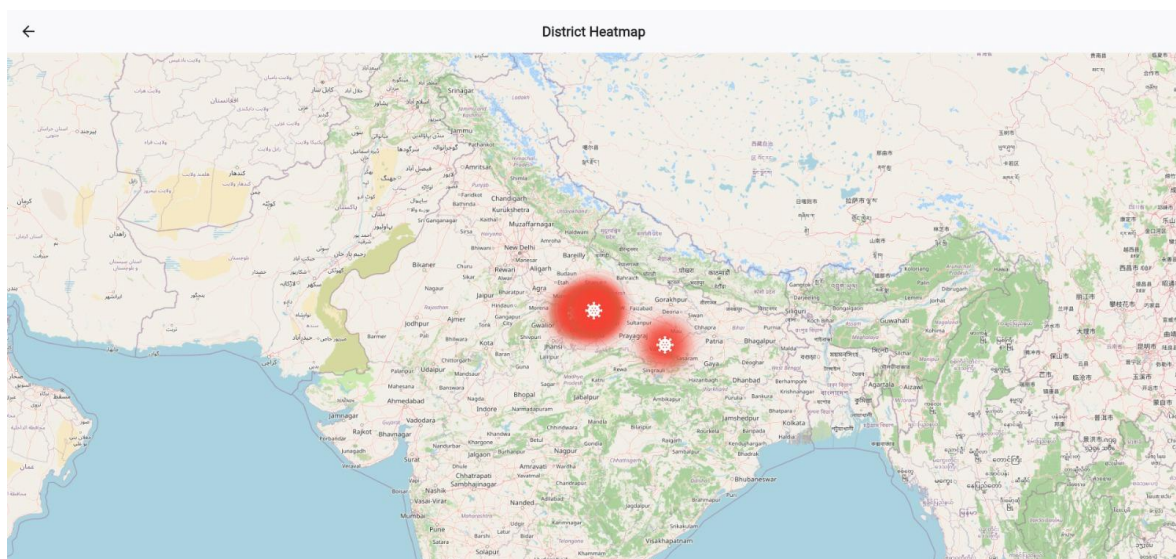


Figure 11: Heatmap

admin_dashboard_screen.dart

```
Future<void> _registerWorker() async {  
  if (!_formKey.currentState!.validate()) return;  
  
  setState(() => _isLoading = true);  
  
  try {  
    final response = await http.post(  
      Uri.parse('${ApiConfig.baseUrl}/admin/workers'),  
      headers: {'Content-Type': 'application/json'},  
      body: jsonEncode({  
        'firstName': _firstNameController.text.trim(),  
        'lastName': _lastNameController.text.trim(),  
        'village': _villageController.text.trim(),  
        'phone': _mobileController.text.trim(),  
        'age': int.tryParse(_ageController.text.trim()),  
        'block': _blockController.text.trim(),  
      })),  
    );  
  
    if (response.statusCode == 200) {  
      final newWorker = jsonDecode(response.body);  
  
      setState(() {  
        _generatedWorker = newWorker;  
        // Clear form  
        _firstNameController.clear();  
        _lastNameController.clear();  
      });  
    }  
  }  
}
```

```

    _villageController.clear();
    _mobileController.clear();
    _ageController.clear();
    _blockController.clear();
  });

  if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(
        content: Text('Worker Provisioned Successfully'),
        backgroundColor: Colors.green,
      ),
    );

    // Call the callback to refresh the workers table
    if (widget.onWorkerCreated != null) {
      widget.onWorkerCreated!();
    }
  }
} else {
  throw Exception('Failed to register worker');
}
} catch (e) {
  if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text('Error: $e'),
        backgroundColor: Colors.red,

```

```

    ),
  );
}
} finally {
  if (mounted) {
    setState(() => _isLoading = false);
  }
}
}
}

```

Admin Dashboard						
Active ASHA Personnel						
Manage and monitor registered ASHA workers						
Registered Workers (9)						
ASHA ID	Full Name	Age	Assigned Village	Block	Mobile	Secure PIN
SAK744	Sakshi Dhoni	25	Ranchi	Ranchi Block	9876543210	6YN0Ncgei0KoAe2yUgnGTIZdr/HkIIFVRIw8F4e8/IE=
RIT146	Ritika Sharma	28	Mumbai	Mumbai Block	9876543211	quQC2zX3B811DuTaqlYUd0Aco1CrkzEsfd1d78KHD54=
SON448	Sonpari Chauhan	30	Sonbhadra	Sonbhadra Block	9876543212	fIIFKTrIOIEeOmEOXQ6ftJtamJUbH5n89r0Mn2F4tdo=
KAV354	Kavya Patel	26	Ahmedabad	Ahmedabad Block	9876543213	73l8gRjwLftklgfdXT+MdIMEJjwGPVMsyVxe16iYpk8=
RIY951	Riya Singh	88	kerla	kerla	8888888888	A6xnQhzb4Vx2HuGI4IXwZ5U2I8IzILRFnhP5eNfIRvQ=
ANU786	Anupma Mathoor	39	Sindh	Shindhu	7777777777	A6xnQhzb4Vx2HuGI4IXwZ5U2I8IzILRFnhP5eNfIRvQ=
KAV125	Kavya bajpai	33	kanpur dehat	kanpur	7894561238	A6xnQhzb4Vx2HuGI4IXwZ5U2I8IzILRFnhP5eNfIRvQ=
GOK718	Goku Sharma	33	www	HH	12333321213	A6xnQhzb4Vx2HuGI4IXwZ5U2I8IzILRFnhP5eNfIRvQ=
MIT093	Mithali Singh	35	Belwa	Oadr	1234567890	A6xnQhzb4Vx2HuGI4IXwZ5U2I8IzILRFnhP5eNfIRvQ=

Figure 12: Asha Personnel

Register ASHA Worker

First Name

Last Name

Village

Mobile Number

Age

Block / Tehsil

Cancel

Generate & Register

Figure 13: ASHA Worker Register

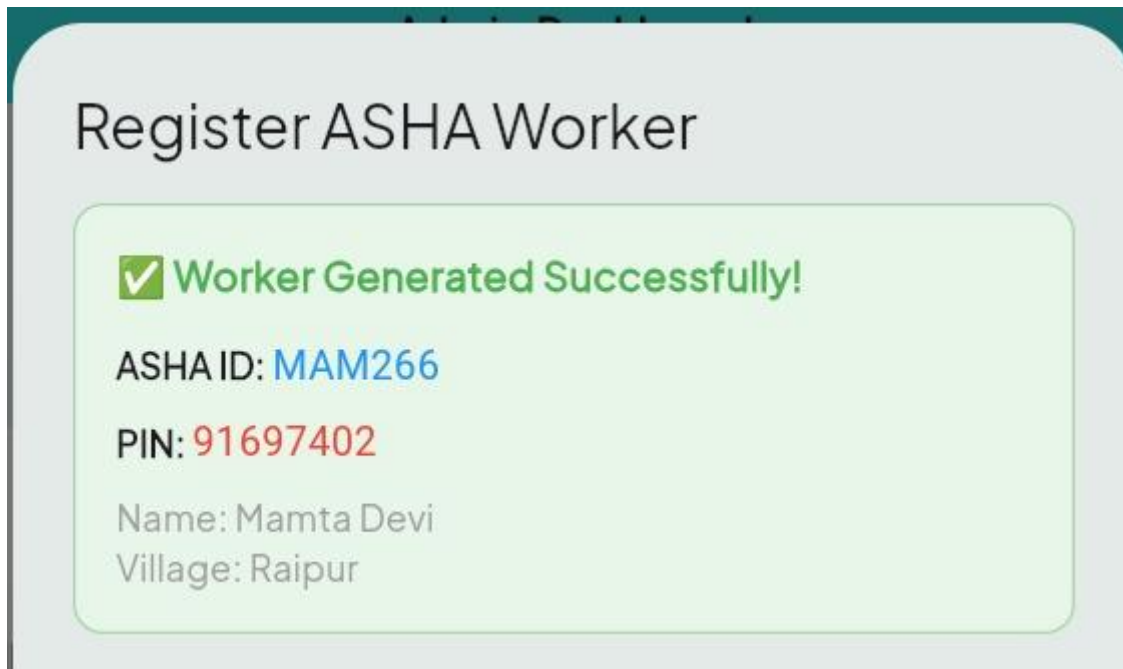


Figure 14: ASHA ID With OTP

```
// Form Controllers

final _firstNameController = TextEditingController();
final _lastNameController = TextEditingController();
final _villageController = TextEditingController();
final _mobileController = TextEditingController();
final _ageController = TextEditingController();
final _blockController = TextEditingController();

// Registration Method

Future<void> _registerWorker() async {
  if (!_formKey.currentState!.validate()) return;
  setState(() => _isLoading = true);

  try {
    final response = await http.post(
      Uri.parse('${ApiConfig.baseUrl}/admin/workers'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'firstName': _firstNameController.text.trim(),
        'lastName': _lastNameController.text.trim(),
        'village': _villageController.text.trim(),
        'phone': _mobileController.text.trim(),
```

```
'age': int.tryParse(_ageController.text.trim()),
        'block': _blockController.text.trim(),
      })),
    );

    if (response.statusCode == 200) {
      final newWorker = jsonDecode(response.body);

      setState(() {
        _generatedWorker = newWorker; // Contains ashaId and PIN

        // Clear form fields
        _firstNameController.clear();
        _lastNameController.clear();
        _villageController.clear();
        _mobileController.clear();
        _ageController.clear();
        _blockController.clear();
      });

      // Show success message
```

```

ScaffoldMessenger.of(context).showSnackBar(

  const SnackBar(

    content: Text('Worker Provisioned Successfully'),

    backgroundColor: Colors.green,

  ),

);

if (widget.onWorkerCreated != null) {

  widget.onWorkerCreated!();

}

} else {

  throw Exception('Failed to register worker');

}

} catch (e) {

  ScaffoldMessenger.of(context).showSnackBar(

    SnackBar(

      content: Text('Error: $e'),

      backgroundColor: Colors.red,

    ),

  );

} finally {

  setState(() => _isLoading = false);

}

}

TextField(

  controller: _firstNameController,

  validator: (v) => v?.isEmpty ?? true ? 'Required' : null,

  decoration: InputDecoration(

    labelText: 'First Name',

    prefixIcon: Icon(Icons.person_outline),

  ),

),

ElevatedButton(

  onPressed: _isLoading ? null : _registerWorker,

  child: _isLoading

    ? CircularProgressIndicator()

    : Text('Generate & Register'),

)

```

BACKEND RESPONSE:

```

{

  "ashId": "MAM266",

  "pin": "91697402",

  "name": "Mamta Devi",

  "assignedVillage": "Raipur",

  "phone": "9839390193",

  "age": 35,

  "block": "Raipur"

}

```

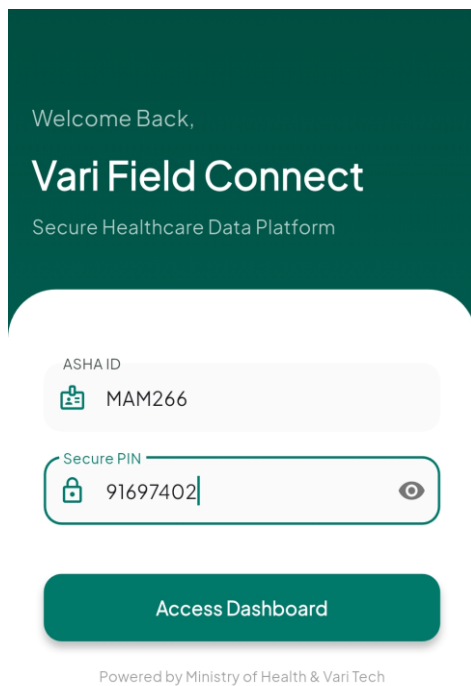


Figure 15: Login Page

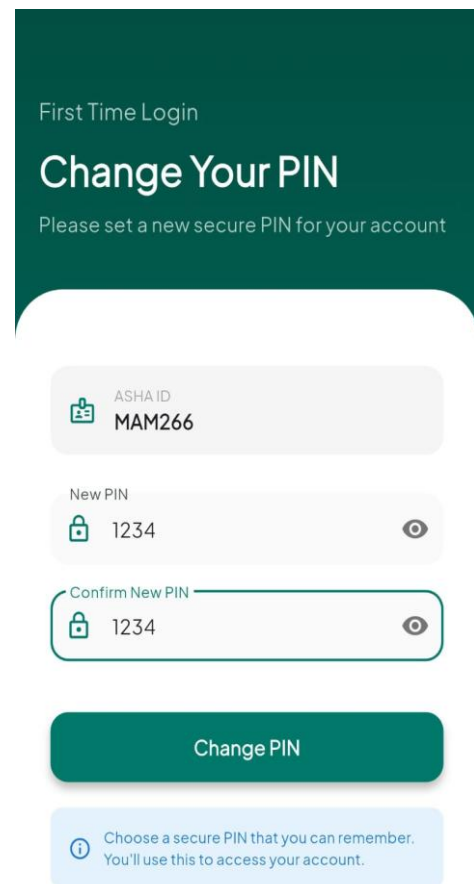


Figure 16: Password Generation

```
// POST /auth/login
final response = await http.post(
  Uri.parse('${ApiConfig.baseUrl}/auth/login'),
  body: jsonEncode({'ashaId': ashaId, 'pin': pin}),
);
// Save to SharedPreferences: ashaId, workerName,
assignedVillage
// Navigate to ChangePassword if firstTimeLogin, else
Dashboard
```

```
// POST /auth/change-password
await http.post(
  Uri.parse('${ApiConfig.baseUrl}/auth/change-
password'),
  body: jsonEncode({'ashaId': ashaId, 'newPin':
newPin}),
);
// Navigate to Dashboard
```

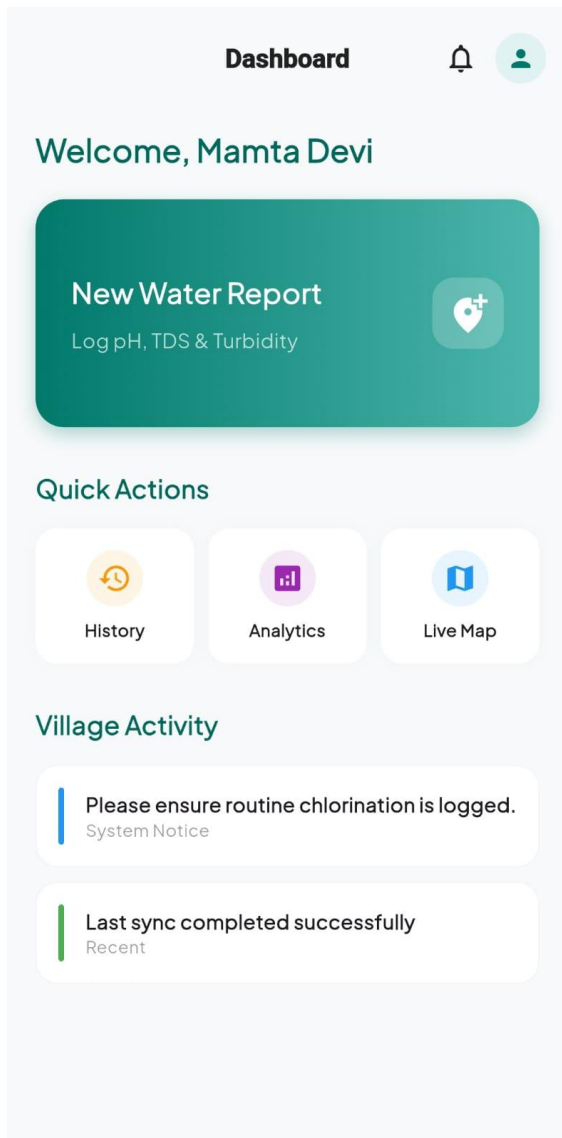


Figure 17: User Dashboard

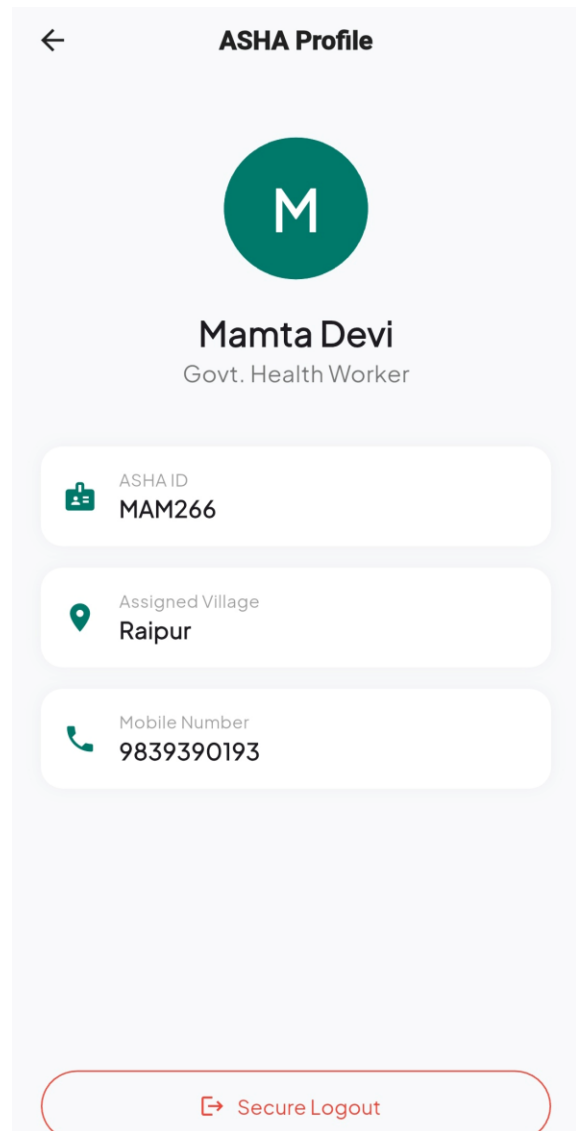


Figure 18: User Profile

```
// Load from SharedPreferences

final prefs = await
SharedPreferences.getInstance();

workerName = prefs.getString('workerName')
?? 'ASHA Worker';

assignedVillage =
prefs.getString('assignedVillage') ?? 'Village';
```

```
// Load profile from SharedPreferences
_ashaId = prefs.getString('ashaId');
_name = prefs.getString('workerName');
_village = prefs.getString('assignedVillage');
_phone = prefs.getString('workerPhone');

// Logout
await prefs.clear();

Navigator.pushReplacement(context, LoginScreen());
```

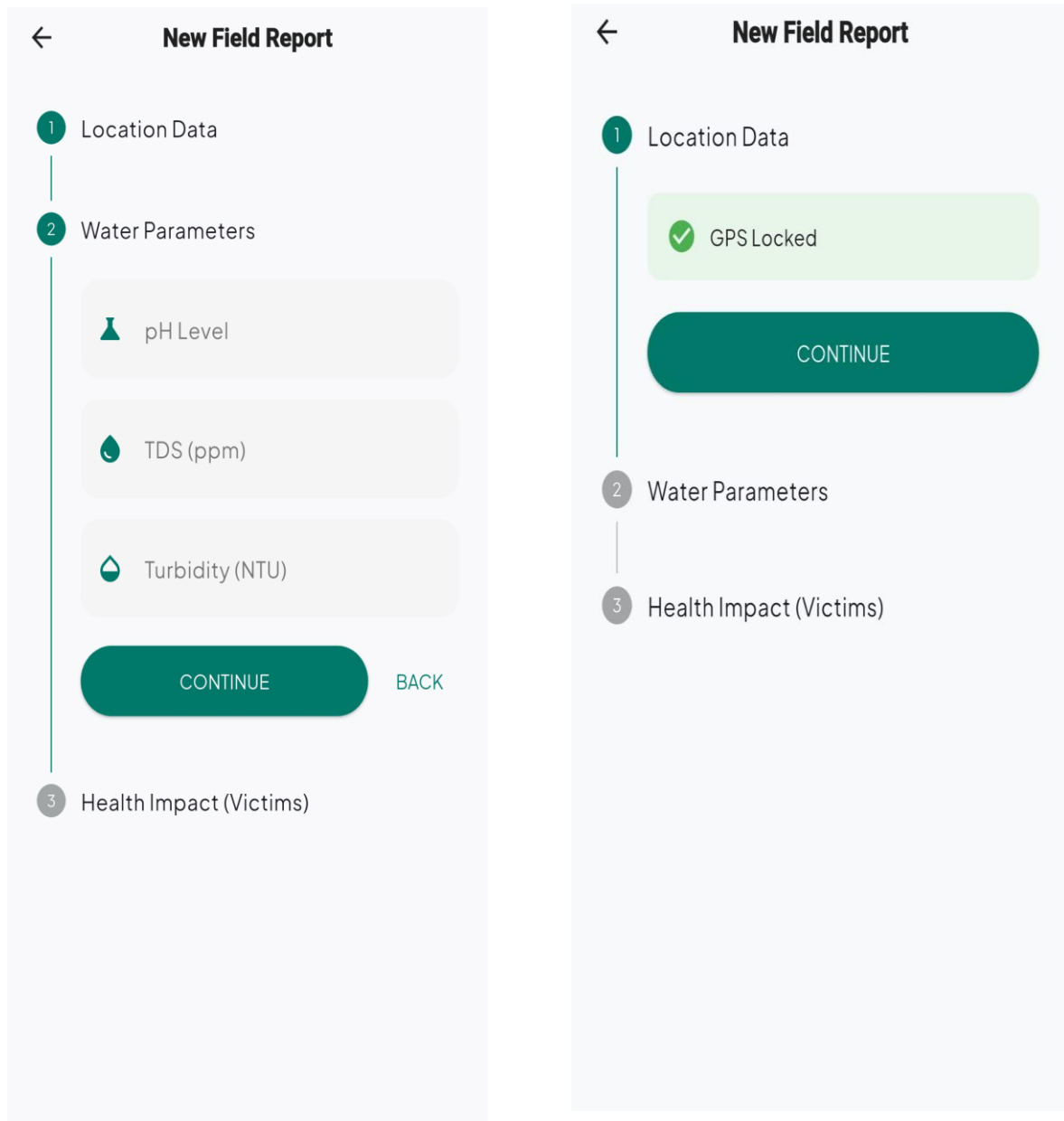


Figure 19: Filling of Water Impurity Data

```
Position position = await Geolocator.getCurrentPosition(
  desiredAccuracy: LocationAccuracy.high,
);
currentLat = position.latitude;
currentLong = position.longitude;
CustomTextField(label: "pH Level", controller: phController),
CustomTextField(label: "TDS (ppm)", controller: tdsController),
CustomTextField(label: "Turbidity (NTU)", controller: turbController),
```


Patient Details

Tap to take photo

Full Name

Age Weight (kg)

Contact Number

Days since symptoms ...

Gender
Male

Condition
Diarrhea

Prior Medication? ☐

Cancel Save Patient

Figure 20: Patient Details

Report History

Report #31
Lat: 26.5509787, Long: 80.2242888
Status: UNSAFE
1 Patients Linked (Tap to View)

Report #28
Lat: 26.5496335, Long: 80.2269787
Status: UNSAFE
1 Patients Linked (Tap to View)

Report #24
Lat: 26.5496522, Long: 80.2269607
Status: UNSAFE
1 Patients Linked (Tap to View)

Report #23
Lat: 26.5485983, Long: 80.2266467
Status: UNSAFE
1 Patients Linked (Tap to View)

Figure 21: Report History

```
victimsForBackend = _victims.map((v) {
  v.remove('patientImageUrl'); return v;
}).toList();

// POST /reports
await http.post(
  Uri.parse('${ApiConfig.baseUrl}/reports'),
  body: jsonEncode({
    "ashaId": _ashaId, "latitude": lat, "longitude": long,
    "phLevel": ph, "tdsLevel": tds, "turbidity": turb,
    "status": status, "victims": victimsForBackend,
  }
),
```

CHAPTER 5: CONCLUSION AND FUTURE WORK

Vari successfully demonstrates the feasibility of a low-cost, mobile-first epidemiological early-warning system designed for deployment with community health workers in rural settings. By co-locating water quality measurements and patient health records at the same GPS coordinate and processing them through an automated safety classification engine, Vari provides a capability that no existing ASHA worker support tool currently offers: the real-time, map-based identification of contaminated water sources and their associated health impact.

The system has been built on a production-grade technology stack—Flutter for cross-platform mobile development, Spring Boot 3.2 with Java 17 for backend services, PostgreSQL for relational data persistence, and Render for cloud deployment—and has been deployed to a live environment accessible over the internet at <https://vari-backend.onrender.com>. The backend architecture demonstrates clean separation of concerns across controller, service, and repository layers, and the database schema correctly models the one-to-many relationship between environmental reports and patient victim records using JPA cascade semantics and bidirectional entity relationships.

The security design, while appropriate for an MVP, establishes a baseline through SHA-256 hashed credential storage, enforced first-login PIN changes, and secure session management via `SharedPreferences`. The administrative worker provisioning module provides a practical mechanism for health programme managers to onboard new ASHA workers with auto-generated unique IDs and secure PINs without exposing database credentials. The entire backend is containerised using Docker and deployable on any Docker-compatible platform, with automated CI/CD through Render's GitHub integration.

The frontend Flutter application provides an intuitive, step-based workflow for field data collection, including GPS-based location tracking, water parameter entry (pH, TDS, turbidity), and comprehensive patient data capture with medical history and photographic documentation. The admin dashboard offers real-time monitoring of field reports, ASHA worker management, and geospatial visualization of contamination zones through an integrated heatmap interface.

In summary, Vari validates the core hypothesis that a lightweight, open-source, cross-platform health-tech stack can deliver meaningful epidemiological surveillance capability at the village level, placing actionable information in the hands of the health worker closest to the problem. The system bridges the critical gap between environmental monitoring and public health response, enabling data-driven decision-making for waterborne disease prevention in resource-constrained settings.

5.1 Future Scope

Several extensions to the Vari platform are identified as high-priority areas for future development:

1. AI/ML-based Outbreak Prediction

Integration with Google Gemini or a locally trained classification model to analyse historical patterns of water quality degradation and disease incidence, generating predictive risk scores for geographic zones before a full outbreak manifests.

2. Real-time Heatmap Dashboard

A web-based geospatial heatmap for district health officers, overlaying Safe/Unsafe zones, disease incidence density, and temporal trends on an interactive map powered by OpenStreetMap or Google Maps API.

3. Push Notification Alerts

When an Unsafe zone is created or when a cluster of victim records is detected within a geographic radius, automated push notifications should be dispatched to the supervising health officer and the relevant ASHA workers in the area.

4. HMIS Integration

An outbound integration layer to automatically forward structured report data to the national Health Management Information System (HMIS) and Integrated Disease Surveillance Programme (IDSP) portals, eliminating double-entry for ASHA workers.

5. JWT-based Authentication

Migration from PIN-based simple authentication to JSON Web Token (JWT) stateless authentication, providing token expiry, refresh capability, and role-based access control (RBAC) for worker, supervisor, and admin roles.

6. Offline-First Capability

Implementation of a local SQLite store in Flutter to queue reports when field workers are in areas with no network coverage, with background synchronisation once connectivity is restored.

7. IoT Sensor Integration

An optional companion mode where pH and TDS sensor readings are transmitted via Bluetooth Low Energy (BLE) directly from a portable water testing device to the Flutter application, eliminating manual parameter entry and reducing measurement error.

REFERENCES

- [1] Adu-Manu, K. S., Adam, N., Tapparello, C., Ayatse, H., & Heinzelman, W. (2017). Energy-harvesting wireless sensor networks: Survey and challenges. *ACM Transactions on Sensor Networks*, 14(2), 1–50.
- [2] Garg, C. C., Bhatt, P., Karan, A., & Mohan, P. (2019). Status of integrated disease surveillance in India: An analysis. *Indian Journal of Community Medicine*, 44(3), 214–219.
- [3] Labrique, A. B., Vasudevan, L., Kochi, E., Fabricant, R., & Mehl, G. (2013). mHealth innovations as health system strengthening tools: 12 common applications and a visual framework. *Global Health: Science and Practice*, 1(2), 160–171.
- [4] Moore, S. M., Lessler, J., Azman, A., & Bhatt, S. (2018). Population-wide cholera vaccination in the Democratic Republic of Congo. *The Lancet Infectious Diseases*, 18(8), 843–852.
- [5] Raj, A., Bhattacharya, B., & Bhattacharya, S. (2021). Community health workers and digital health: A review of the literature. *Journal of Global Health*, 11, 04030.
- [6] Spring Framework Documentation. (2024). Spring Boot 3.2 Reference Guide. <https://docs.spring.io/spring-boot/docs/3.2.x/reference/html/>
- [7] Flutter Documentation. (2024). Flutter 3+ Docs: Get Started. <https://docs.flutter.dev/>
- [8] World Health Organization. (2022). Guidelines for Drinking-Water Quality, 4th edition incorporating the 1st and 2nd addenda. WHO Press, Geneva.
- [9] Bureau of Indian Standards. (2012). IS 10500:2012 — Drinking Water Specification (Second Revision). BIS, New Delhi.
- [10] Supabase. (2024). Supabase Documentation: Database. <https://supabase.com/docs/guides/database>
- [11] PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/>
- [12] Hibernate ORM. (2024). Hibernate 6 User Guide. <https://hibernate.org/orm/documentation/6.0/>
- [13] Docker Inc. (2024). Docker Documentation. <https://docs.docker.com/>
- [14] Render.com. (2024). Render Documentation: Web Services. <https://render.com/docs/web-services>
- [15] Google Developers. (2024). Dart Programming Language Documentation. <https://dart.dev/guides>